

FlashQLA: 面向 Qwen 系列模型线性注意力的高性能优化实践

演讲人: 章程瑞东

阿里巴巴 Qwen 团队算法工程师

AiCon

全球人工智能开发与应用大会

极客邦科技 2026 年会议规划

促进软件开发及相关领域知识与创新的传播



参会咨询

InfoQ 技术大会

QCon

上海站

全球软件开发大会

大模型正在重新定义软件

Large Language Model Is Redefining
The Software

10月22-24日 / 上海建工浦江皇冠假日酒店

· AI Agent

· Vibe Coding

· 智能可观测

· 推理基建

· 模型攻防

· AI x 创造力

InfoQ 技术大会

AiCon

北京站

全球人工智能开发与应用大会

探索 AI 应用边界

Explore the limits of AI applications

12月18-19日 / 北京

· 大模型架构创新

· 多模态 AI 产业融合

· 具身智能

· AI for Science

· 大模型安全

目录

01 定位与背景

02 核心技术原理

03 性能与基准

04 工程与生态

05 场景与展望

06 致谢

01 定位与背景

FlashQLA 为何而生

从 Softmax 注意力到 GDN 线性注意力

01

$O(N^2)$ 平方级

Softmax 注意力

$$\begin{aligned} P &\leftarrow \text{Tril}(QK^T) \\ P &\leftarrow \text{softmax}(P) \\ O &\leftarrow PV \end{aligned}$$

- $O(N^2)$ 计算开销
- $O(N)$ KV cache

02

$O(N)$ 线性级

线性注意力

$$\begin{aligned} P &\leftarrow \text{Tril}(QK^T) \\ P &\leftarrow \text{softmax}(P) \\ O &\leftarrow PV \end{aligned}$$



$$\begin{aligned} S_{t+1} &\leftarrow S_t + k_t^T v_t \\ o_t &\leftarrow q_t S_t \end{aligned}$$

- 计算复杂度线性化
- 以固定大小状态替代显式 KV

03

表达力更强的线性注意力

Gated DeltaNet (GDN)

$$\begin{aligned} S_{t+1} &\leftarrow a_t S_t (I - \beta_t k_t k_t^T) \\ &\quad + \beta_t k_t^T v_t \\ o_t &\leftarrow q_t S_t \end{aligned}$$

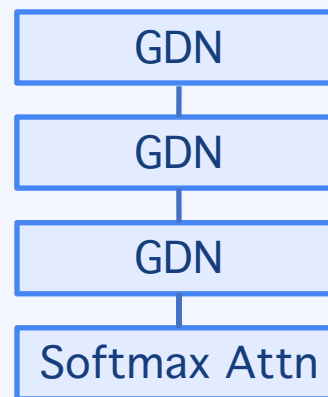
- 更新门 $\beta_t \in (0,1)$
- 遗忘门 $a_t \in (0,1)$

兼顾表达力与效率

04

混合架构

Qwen 3.5-3.8



■ GDN Chunked Prefill: 前向四步流程

① Chunk 内逆矩阵 A

$$A_i \leftarrow \left(I + \text{StrictTril}(\text{Diag}(\beta_i)(\Gamma_i \odot K_i K_i^\top)) \right)^{-1}$$

② 中间投影 W / U

$$\begin{aligned} W_i &\leftarrow A_i \text{Diag}(\beta_i) \text{Diag}(\gamma_i) K_i \\ U_i &\leftarrow A_i \text{Diag}(\beta_i) V_i \end{aligned}$$

③ 状态递推 V' / S

$$\begin{aligned} V'_i &\leftarrow U_i - W_i S_i \\ S_{i+1} &\leftarrow \gamma_{i,C-1} S_i + K_i^\top \text{Diag}\left(\frac{\gamma_{i,C-1}}{\gamma_i}\right) V'_i \end{aligned}$$

④ Chunk 输出 O

$$O_i \leftarrow \text{Diag}(\gamma_i) Q_i S_i + \text{Tril}(\Gamma_i \odot Q_i K_i^\top) V'_i$$

① FLA 中 每步对应一个独立 Triton kernel; $W_i U_i S_i$ 为中间变量, 需写回 HBM; γ_i 为 chunk 内 gate 累乘。

■ 现有实现的痛点：SM 利用率与 Kernel 开销



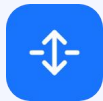
01

并行度不足

张量并行切分头数、长序列、小头数模型下，可调度 block 数不足以填满 GPU 全部 SM。

SSM state 递推 kernel 的 thread block 数仅为 $\text{batch_size} \times \text{num_heads}$

SM 利用率低



02

状态传递开销

Memory-bound kernel 串联，中间状态需反复读写，性能瓶颈位于 HBM 访存。

FLA 四步流程每步一个 kernel、反复读取 K/V；中间变量 $W / U / S$ 需写回 HBM 再由下一个 kernel 读取。

Tensor Core 算力利用率低



03

Triton Kernel 生成上限

Triton 自动生成的调度难以精细控制 TMA 拷贝、共享内存与 warp 分工，压不满 Hopper / Blackwell 硬件的 Tensor Core 与异步流水。

自动调优触不到硬件峰值

- Fully-fused kernel 可消除中间结果往返 HBM，但使得全部计算受限于递推并行度 —— 访存与并行度的解法相互矛盾。

基于 TileLang 的 GDN Chunked Prefill 前/反向算子库



定位

GDN Chunked Prefill 融合 Kernel 库

- ✓ 兼顾 **Tensor Core** 利用率与 **SM** 利用率的方案
- ✓ 基于 TileLang 实现
- ✓ 前向 + 反向同时支持



开源 · 生态

MIT 许可,即装即用

```
pip install flash-qla
```

- ✓ Flash-Linear-Attention 官方 GDN 后端



性能 · 硬件

训练与推理均受益

2-3x 前向提速

2x 反向提速

- ✓ 支持 Hopper /Blackwell (SM90+)

02 核心技术原理

AutoCP · 代数重写 · Warp-Specialization

GDN 的序列并行方案

① 如果从错误的初始状态开始递推，怎样校正？

$$S_1 = Y_{0,C-1}S_0 + K_0^T \text{Diag}\left(\frac{Y_{0,C-1}}{Y_0}\right)(U_0 - W_0S_0)$$
$$S_1^* = Y_{0,C-1}S_0^* + K_0^T \text{Diag}\left(\frac{Y_{0,C-1}}{Y_0}\right)(U_0 - W_0S_0^*)$$



$$S_1 - S_1^*$$
$$= Y_{0,C-1}(S_0 - S_0^*) - K_0^T \text{Diag}\left(\frac{Y_{0,C-1}}{Y_0}\right)W_0(S_0 - S_0^*)$$



$$S_{i+1} - S_{i+1}^*$$
$$= \left[Y_{i,-1}I - K_i^T \text{Diag}\left(\frac{Y_{i,-1}}{Y_i}\right)W_i \right] (S_i - S_i^*)$$

② 每个 CP rank 只需一个校正矩阵

$$S_{j,0}^* \leftarrow 0, M_{j,0} \leftarrow I$$
$$S_{j,i+1}^* \leftarrow Y_{i,C-1}S_{j,i}^* + \dots$$
$$M_{j,i+1} \leftarrow \left[Y_{j,i,C-1}I - K_{j,i}^T \text{Diag}\left(\frac{Y_{j,i,C-1}}{Y_{j,i}}\right)W_{j,i} \right] M_{j,i}$$

$$S_{j+1,0} = S_{j,L} \leftarrow S_{j,L}^* + M_{j,L}S_{j,0}$$

③ 三步实现序列并行

parallel for j:
calc $S_{j,L}^*, M_{j,L}$



for j:
calc $S_{j,0}$



parallel for j:
calc $S_{j,i}, O_{j,i}$

支持序列并行的有限算子融合

$$A_i \leftarrow \left(I + \text{StrictTril}(\text{Diag}(\beta_i)(\Gamma_i \odot K_i K_i^\top)) \right)^{-1}$$

$$W_i \leftarrow A_i \text{Diag}(\beta_i) \text{Diag}(\gamma_i) K_i$$
$$U_i \leftarrow A_i \text{Diag}(\beta_i) V_i$$

$$V'_i \leftarrow U_i - W_i S_i$$
$$S_{i+1} \leftarrow \gamma_{i,C-1} S_i + K_i^\top \text{Diag}\left(\frac{\gamma_{i,C-1}}{\gamma_i}\right) V'_i$$

$$O_i \leftarrow \text{Diag}(\gamma_i) Q_i S_i + \text{Tril}(\Gamma_i \odot Q_i K_i^\top) V'_i$$

$$A_i \leftarrow \left(I + \text{StrictTril}(\text{Diag}(\beta_i)(\Gamma_i \odot K_i K_i^\top)) \right)^{-1}$$

$$S_{j,i+1}^* \leftarrow \gamma_{i,C-1} S_{j,i}^* + \dots$$
$$M_{j,i+1} \leftarrow [\dots] M_{j,i}$$
$$S_{j+1,0} \leftarrow S_{j,L}^* + M_{j,L} S_{j,0}$$

$$W_i \leftarrow \dots, U_i \leftarrow \dots, V'_i \leftarrow \dots$$
$$S_{i+1} \leftarrow \gamma_{i,C-1} S_i + K_i^\top \text{Diag}\left(\frac{\gamma_{i,C-1}}{\gamma_i}\right) V'_i$$
$$O_i \leftarrow \text{Diag}(\gamma_i) Q_i S_i + \text{Tril}(\Gamma_i \odot Q_i K_i^\top) V'_i$$

AutoCP: 自动化卡内序列并行

三步序列并行方案的耗时估计

- N = 总 chunk 数量
- L = 单 CP rank 上的 chunk 数量

parallel for j :
calc $S_{j,L}^*, M_{j,L}$ $\Rightarrow$ for j :
calc $S_{j,0}$ $\Rightarrow$ parallel for j :
calc $S_{j,i}, O_{j,i}$

递推长度 $\propto L$ 递推长度 $\propto N/L$ 递推长度 $\propto L$

1-wave 简化模型

$$t = \lambda_1 L + \lambda_2 \frac{N}{L} \Rightarrow \operatorname{argmin}_t L = \lambda \sqrt{N}$$

- λ 由 kernel 性能决定。

AutoCP 的启用条件

- 总头数（相对于硬件 SM 数）足够小
- 序列足够长

$$\text{batch_size} \times \text{num_heads} \leq 40$$

或

$$\leq 56 \text{ 且 } \text{seq_len} \geq 8192$$

- 满足条件时自动启用卡内序列并行

GDN 的滑动窗口性质

1 GDN 递推

$$\downarrow S_{t+1} \leftarrow a_t S_t (I - \beta_t k_t k_t^T) + \beta_t k_t^T v_t \quad a_t \in (0,1)$$

2 指数衰减

$\downarrow$ 每个 S_t 对后续状态的影响 **指数衰减** $\Rightarrow$ 滑动窗口性质

3 窗口即精确初始化

无需从 S_0 递推：窗口 W 足够长时，
从 $S_{t-W} = 0$ 开始递推即可获得 **精确 S_t** ，即 **warmup**

60–80%

可覆盖的线性注意力头
比例

衰减头无需全序列递推：小窗口 warmup 即可

60–80% 的头

a_t 不恒为 1
属于可窗口化的衰减头

6–8 个 chunk

warmup 即可把误差压到
噪声以下

耗时可忽略

由轻量 gate 统计 kernel
决定 warmup 长度

舍弃 M 矩阵

衰减头无需再计算 M 矩阵

■ 硬件友好的代数改写：提取门控矩阵

🕒 SFU 负载问题：计算 Γ_i 需要 C^2 次 $\exp()$

- γ_i 是 a_i 的累乘：存储 $\log a_i$ 以加代乘提升精度
- 累加后获取 γ_i 需要 $\exp()$ ： $\gamma_i = \exp(g_i)$
- $\Gamma_i = \text{Tril}\left(\text{Diag}(\gamma_i) / \text{Diag}\left(\frac{1}{\gamma_i}\right)\right)$ 直接计算存在数值溢出风险
- 无溢出方案： $\Gamma_{i,s,t} \leftarrow \exp(g_{i,s} - g_{i,t}), s \geq t$

☑ 矩阵求逆中的 Γ_i 可被提取

$$\begin{aligned} A_i &= \left(I + \text{ST}(\beta_i(\Gamma_i \odot K_i K_i^T)) \right)^{-1} = \left(\Gamma_i \odot \left(I + \text{ST}(\beta_i K_i K_i^T) \right) \right)^{-1} \\ &= \left(\gamma_i(\dots) \frac{1}{\gamma_i} \right)^{-1} = \gamma_i(\dots)^{-1} \frac{1}{\gamma_i} = \Gamma_i \odot \left(I + \text{ST}(\beta_i K_i K_i^T) \right)^{-1} \end{aligned}$$

1	0	0	0	0	0	0	0
$\frac{\gamma_{i,1}}{\gamma_{i,0}}$	1	0	0	0	0	0	0
$\frac{\gamma_{i,2}}{\gamma_{i,0}}$	$\frac{\gamma_{i,2}}{\gamma_{i,1}}$	1	0	0	0	0	0
$\frac{\gamma_{i,3}}{\gamma_{i,0}}$	$\frac{\gamma_{i,3}}{\gamma_{i,1}}$	$\frac{\gamma_{i,3}}{\gamma_{i,2}}$	1	0	0	0	0
$\frac{\gamma_{i,4}}{\gamma_{i,0}}$	$\frac{\gamma_{i,4}}{\gamma_{i,1}}$	$\frac{\gamma_{i,4}}{\gamma_{i,2}}$	$\frac{\gamma_{i,4}}{\gamma_{i,3}}$	1	0	0	0
$\frac{\gamma_{i,5}}{\gamma_{i,0}}$	$\frac{\gamma_{i,5}}{\gamma_{i,1}}$	$\frac{\gamma_{i,5}}{\gamma_{i,2}}$	$\frac{\gamma_{i,5}}{\gamma_{i,3}}$	$\frac{\gamma_{i,5}}{\gamma_{i,4}}$	1	0	0
$\frac{\gamma_{i,6}}{\gamma_{i,0}}$	$\frac{\gamma_{i,6}}{\gamma_{i,1}}$	$\frac{\gamma_{i,6}}{\gamma_{i,2}}$	$\frac{\gamma_{i,6}}{\gamma_{i,3}}$	$\frac{\gamma_{i,6}}{\gamma_{i,4}}$	$\frac{\gamma_{i,6}}{\gamma_{i,5}}$	1	0
$\frac{\gamma_{i,7}}{\gamma_{i,0}}$	$\frac{\gamma_{i,7}}{\gamma_{i,1}}$	$\frac{\gamma_{i,7}}{\gamma_{i,2}}$	$\frac{\gamma_{i,7}}{\gamma_{i,3}}$	$\frac{\gamma_{i,7}}{\gamma_{i,4}}$	$\frac{\gamma_{i,7}}{\gamma_{i,5}}$	$\frac{\gamma_{i,7}}{\gamma_{i,6}}$	1

■ 硬件友好的代数改写：提取门控矩阵

⊖ 重写前 原始路径

$$A_i \leftarrow \left(I + \text{StrictTril}(\text{Diag}(\beta_i)(\Gamma_i \odot K_i K_i^\top)) \right)^{-1}$$

$$W_i \leftarrow A_i \text{Diag}(\beta_i) \text{Diag}(\gamma_i) K_i$$

$$U_i \leftarrow A_i \text{Diag}(\beta_i) V_i$$

$$S_{i+1} \leftarrow \gamma_{i,C-1} S_i + K_i^\top \text{Diag}\left(\frac{\gamma_{i,C-1}}{\gamma_i}\right) (U_i - W_i S_i)$$

$$O_i \leftarrow \text{Diag}(\gamma_i) Q_i S_i + \text{Tril}(\Gamma_i \odot Q_i K_i^\top) (U_i - W_i S_i)$$

矩阵求逆和递推 **kernel** 都需要计算 Γ_i

⊙ 重写后 等价搬移 · 精度不变

$$A'_i \leftarrow \left(I + \text{StrictTril}(\text{Diag}(\beta_i)(K_i K_i^\top)) \right)^{-1}$$

$$W_i \leftarrow (\Gamma_i \odot A'_i) \text{Diag}(\beta_i) \text{Diag}(\gamma_i) K_i$$

$$U_i \leftarrow (\Gamma_i \odot A'_i) \text{Diag}(\beta_i) V_i$$

$$S_{i+1} \leftarrow \gamma_{i,C-1} S_i + K_i^\top \text{Diag}\left(\frac{\gamma_{i,C-1}}{\gamma_i}\right) (U_i - W_i S_i)$$

$$O_i \leftarrow \text{Diag}(\gamma_i) Q_i S_i + \text{Tril}(\Gamma_i \odot Q_i K_i^\top) (U_i - W_i S_i)$$

⊙ 矩阵求逆 **kernel** 无需计算 Γ_i

■ 硬件友好的代数改写：矩阵连乘优化

① 序列并行转移矩阵

$$\begin{aligned}W_i &\leftarrow A_i \text{Diag}(\beta_i) \text{Diag}(\gamma_i) K_i \\U_i &\leftarrow A_i \text{Diag}(\beta_i) V_i \quad V'_i \leftarrow U_i - W_i S_i \\S_{i+1} &\leftarrow \gamma_{i,C-1} S_i + K_i^\top \text{Diag}\left(\frac{\gamma_{i,C-1}}{\gamma_i}\right) V'_i \\M_{i+1} &\leftarrow \left[\gamma_{i,C-1} I - K_i^\top \text{Diag}\left(\frac{\gamma_{i,C-1}}{\gamma_i}\right) W_i \right] M_i\end{aligned}$$



$$\begin{aligned}X_i &\leftarrow -\text{Diag}(\beta_i) A_i^\top K_i \quad Z_i \leftarrow K_i M_i \\Y_i &\leftarrow \gamma_{i,C-1} K_i S_i - \text{Diag}\left(\frac{\gamma_{i,C-1}}{\gamma_i}\right) V_i \\S_{i+1} &\leftarrow \gamma_{i,C-1} S_i + X_i^\top Y_i \\M_{i+1} &\leftarrow \gamma_{i,C-1} (M_i + X_i^\top Z_i)\end{aligned}$$

② 前向递推

$$\begin{aligned}W_i &\leftarrow A_i \text{Diag}(\beta_i) \text{Diag}(\gamma_i) K_i \\U_i &\leftarrow A_i \text{Diag}(\beta_i) V_i \\V'_i &\leftarrow U_i - W_i S_i\end{aligned}$$



$$V'_i \leftarrow A_i \text{Diag}(\beta_i) (V_i - \text{Diag}(\gamma_i) K_i S_i)$$

③ 反向递推

$$dK \leftarrow dAK + dA^\top K$$



$$dK \leftarrow (dA + dA^\top) K$$

Tilelang Warp-Specialized Kernel



01 · ROOFLINE

计算 vs 访存

18 单位

矩阵乘计算量

30 单位

HBM 访存量

H200 ≈ 1000 TFLOPS $\Rightarrow$ 计算 ~ 9 ns

HBM 4.8 TB/s $\Rightarrow$ 访存 ~ 15 ns

memory-bound $\Rightarrow$ 需重叠遮盖



02 · WASP

Warpgroup 流水分工

WG3 生产者 · TMA 搬运
 $Q_i, K_i, V_i, \beta_i, \gamma_i, A_i'$

WG2 消费者 · K & V
 $V_i' \leftarrow A_i \text{Diag}(\beta_i) (V_i - \text{Diag}(\gamma_i) K_i S_i)$

WG1 消费者 · P & O
 $O_i \leftarrow \text{Diag}(\gamma_i) Q_i S_i + P_i V_i'$

WGO 消费者 · S
 $S_{i+1} \leftarrow \gamma_{i,C-1} S_i + K_i^T \text{Diag} \left(\frac{\gamma_{i,C-1}}{\gamma_i} \right) V_i'$



03 · 资源预算

每 Thread Block 预算

寄存器
 ≤ 65536 平均 128 寄存器/线程
容量紧张

SMEM
228 KB 恰好容纳 ping-pong
双缓冲 + 临时变量

双缓冲 $\Rightarrow$ 遮盖访存、填满计算



Warpgroup 特化: 搬运 / Tensor Core / CUDA Core 并行重叠, 消除流水线气泡

03 性能与基准

前向 2-3x · 反向 2x

■ 核心结果：前向 2~3x / 反向 2x

↑ 前向加速

2-3x

Forward pass vs
FLA Triton kernel



训推通用

训练与推理同
一套内核受益

↑ 反向加速

2x

Backward pass vs
FLA Triton kernel

结论

训练(前向 + 反向)与预填充推理**同时受益**
优化覆盖全链路



实测平台：NVIDIA Hopper / Blackwell

Tilelang Warp-Specialized Kernel

H200 单层前向实测延迟(ms)

场景	FlashQLA(基准)	FLA Triton	FlashInfer	提速(vs FLA / FlashInfer)
397B/122B TP8 1×32K	0.310	0.913	1.653	2.95× / 5.33×
TP8 1×16K	0.184	0.465	0.833	2.53× / 4.53×
TP8 24576+8192 chunked	0.302	0.767	1.242	2.54× / 4.11×
TP4 1×32K	0.486	1.250	1.654	2.57× / 3.40×
27B TP2 1×32K	0.659	1.564	1.616	2.37× / 2.45×
2B TP1 1×32K	0.493	1.285	1.640	2.60× / 3.33×

① 注：TP 并行度越高提速越大（AutoCP 生效）；FlashInfer 不支持反向

■ 基准覆盖与可复现性

🔧 模型配置

Qwen 3.5 - 3.8 家族 GDN head 配置
`num_v_heads = 64 / 48 / 32 / 24 / 16 / 8`
逐一映射 TP1 - TP8 并行规模

🖨️ 硬件平台

Hopper / Blackwell GPU 实测平台
SM90 / SM100 / SM103 / SM120 / SM121
全面覆盖现代算力

🔗 基线版本

FLA	0.5.0	Triton	3.5.1
FlashInfer	0.6.9	TileLang	0.1.9

🛡️ 软件要求

☑️ CUDA	12.8+
☑️ PyTorch	2.8+

☑️ 固定基线版本 + 锁定软硬件栈，全部基准数据均可独立复现

04 工程进展与开源生态

API · FLA 后端 · vLLM 接入

API 设计与框架继承



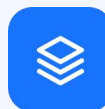
FLA 通用接口约定

```
from qla import chunk_gated_delta_rule
o, final_state = chunk_gated_delta_rule(
    q, k, v, g, beta, initial_state=h0,
    cu_seqlens=..., auto_cp=True)
```

$q, k \in [B, T, Hk, K]$ · $v \in [B, T, Hv, V]$
 $initial_state \in [B, Hv, K, V]$
 $g, beta \in [B, T, Hv]$ · $cu_seqlens$ 支持变长序列

运行要求 SM90+ (Hopper/Blackwell) ·
CUDA 12.8+ · PyTorch 2.8+ ·

`pip install flash-qla`



FLA 官方后端

自 `v0.1.2` 起, 通过标准
`FLA API` 成为
`flash-linear-attention` 的
`GDN` 后端

✓ 已上线



vLLM 集成

`issue #43089` 提议将
`FlashQLA` 作为 `GDN` 提
速后端, 接入推理引擎生
态

🔄 在途

■ 开源演进时间线

★ 644 Stars

MIT 许可



2026-04-24

仓库创建

定位 GDN Chunked Prefill kernel 库



2026-06-30

v0.1.1 发布

反向卡内序列并行 + SM100 支持



2026-07-09

v0.1.2 发布

SM120 前向支持 + 成为 FLA 官方 GDN 后端



在途

v0.1.3

SM120 反向支持 + 卡间序列并行 + AutoCP 新算法

05 应用场景与展望

从预训练到端侧推理

■ 三类应用场景

训练



大模型预训练

2.4T / 397B

122B / 27B

支持 1M 长上下文

端侧推理



端侧 agentic 推理

2B / 0.8B

batch_size=1 & 小模型推理

线上服务



TP 长序列线上部署

长序列

TP 并行线上服务

■ 覆盖范围

模型 0.8B-2.4T

TP1-TP8

变长序列

■ 总结与要点回顾



01

定位

GDN Chunked Prefill
算子库

TileLang 构建
MIT 开源



02

三大技术

AutoCP 卡内序列并行

硬件友好的代数重写

Warp 特化融合 kernel



03

性能结果

前向 **2-3x**

反向 **2x**

(Hopper / Blackwell)



04

开箱即用

pip install flash-qla

FLA 官方 GDN 后端

致谢 Flash Linear Attention / TileLang / FlashInfer / 关于 GDN 序列并行的前序工作

QCon

全球软件开发大会

上海
站

大模型正在重新定义软件

Large Language Model Is Redefining The Software

2026 年 10 月 22-24 日 / 上海建工浦江皇冠假日酒店



参会
咨询

**专题：从 App 到 Agent：字节跳动
Supabase 的 Agent-Native 规模化实践**

专题演讲嘉宾：张国强

字节跳动 / 资深技术专家



**专题：复杂业务 Agent 的持续进化：
快手电商导购的 Harness Loop 实践**

专题演讲嘉宾：包磊

快手 / 资深技术专家



**专题：从 Harness 到 Loop：阿福 Agent
小队如何处理持续涌入的线上 Badcase**

专题演讲嘉宾：肖汉松

蚂蚁集团 / 阿福 Harness 架构组负责人



**专题：AI Coding 在大型客户端工程中的
落地实践——从通才模型到专才 Agent**

专题演讲嘉宾：杨凯

小红书 / 客户端基础技术负责人



**专题：AI 时代的一人公司：从 Idea 到
Growth 的完整 workflow**

专题演讲嘉宾：casa

独立开发



**专题：Agent 驱动的全平台 GUI 持续
验证实践**

专题演讲嘉宾：王政达

微软 / SENIOR SOFTWARE ENGINEER



THANKS

AiCon
全球人工智能开发与应用大会